

SmartBancs App — Documento técnico de arquitectura, desarrollo, operación e IA

1. Resumen ejecutivo

SmartBancs es un MVP de una plataforma bancaria orientada a procesar transferencias en tiempo real y generar recomendaciones financieras mediante un servicio de IA desacoplado.

La solución separa el flujo financiero crítico del procesamiento de recomendaciones. La transferencia se ejecuta dentro de una transacción ACID en PostgreSQL, bloqueando las dos cuentas en un orden determinista para evitar condiciones de carrera y reducir el riesgo de deadlocks. Una vez confirmada la operación, se publica un evento en Redis para que un worker independiente genere la recomendación.

El core legado Bancs no se consulta directamente en cada solicitud de la aplicación. El diseño propone una capa de integración y un modelo operativo desacoplado, con eventos/CDC cuando las interfaces del banco lo permitan y reconciliación periódica contra Bancs.

La solución incorpora logs estructurados, métricas Prometheus, trazabilidad mediante request_id y transaction_id, OpenTelemetry, alertas básicas y un endpoint de journey sintético.

Alcance del benchmark: el objetivo de 10 000 TPS es un requisito de arquitectura y escalabilidad. El entorno local del MVP no certifica esa capacidad. Las pruebas disponibles muestran comportamiento bajo carga controlada y también evidencian el punto de saturación de un entorno agresivo.

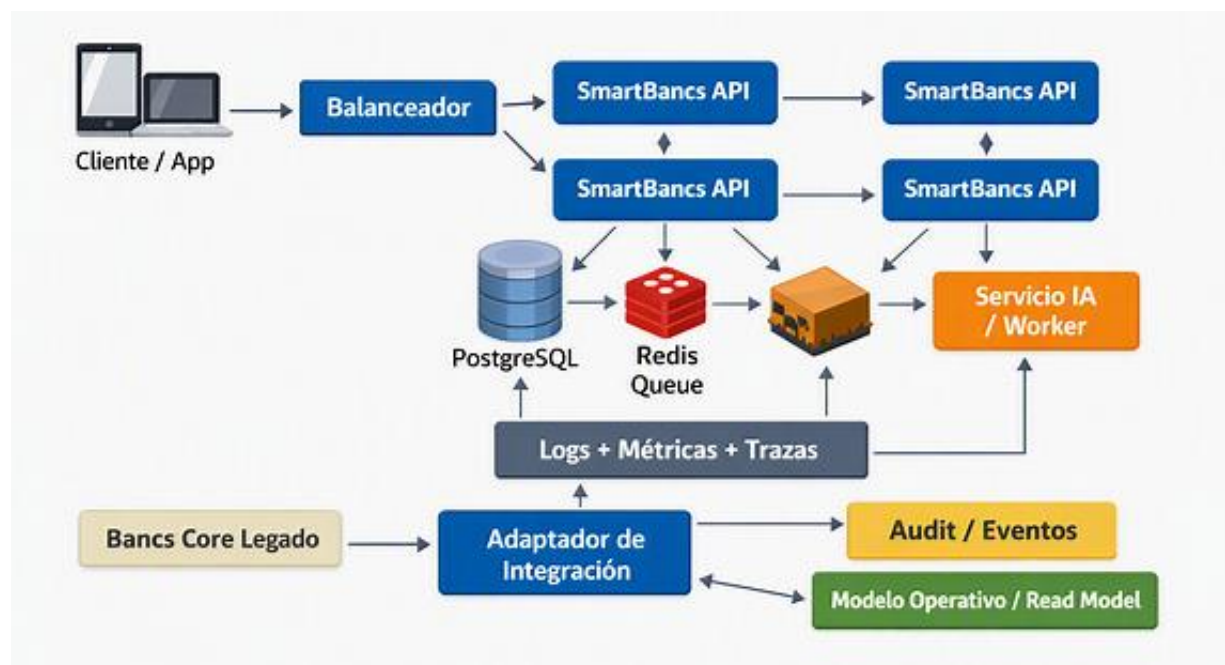
2. Requisitos y respuesta técnica

Requisito	Respuesta técnica
API para transacciones	FastAPI REST mediante POST /api/v1/transfers.

Alta concurrencia	API stateless y escalamiento horizontal con Kubernetes/HPA.
Consistencia	PostgreSQL y transacciones ACID.
Race conditions	SELECT ... FOR UPDATE y orden determinista de locks.
IA no bloqueante	Redis y worker independiente.
Integración Bancs	Adaptador de eventos, normalización y auditoría; diseño productivo con eventos/CDC y reconciliación.
ETL	Python y pandas.
Observabilidad	Logs JSON, Prometheus y OpenTelemetry.
Trazabilidad	request_id y transaction_id.
IaC / ejecución reproducible	Docker Compose, Kubernetes, Terraform y Ansible.
Incidentes	Métricas, consultas de diagnóstico, acciones inmediatas y post-mortem.
Seguridad	Credenciales sintéticas/locales; controles productivos definidos como evolución.

3. Arquitectura

3.1 Vista lógica



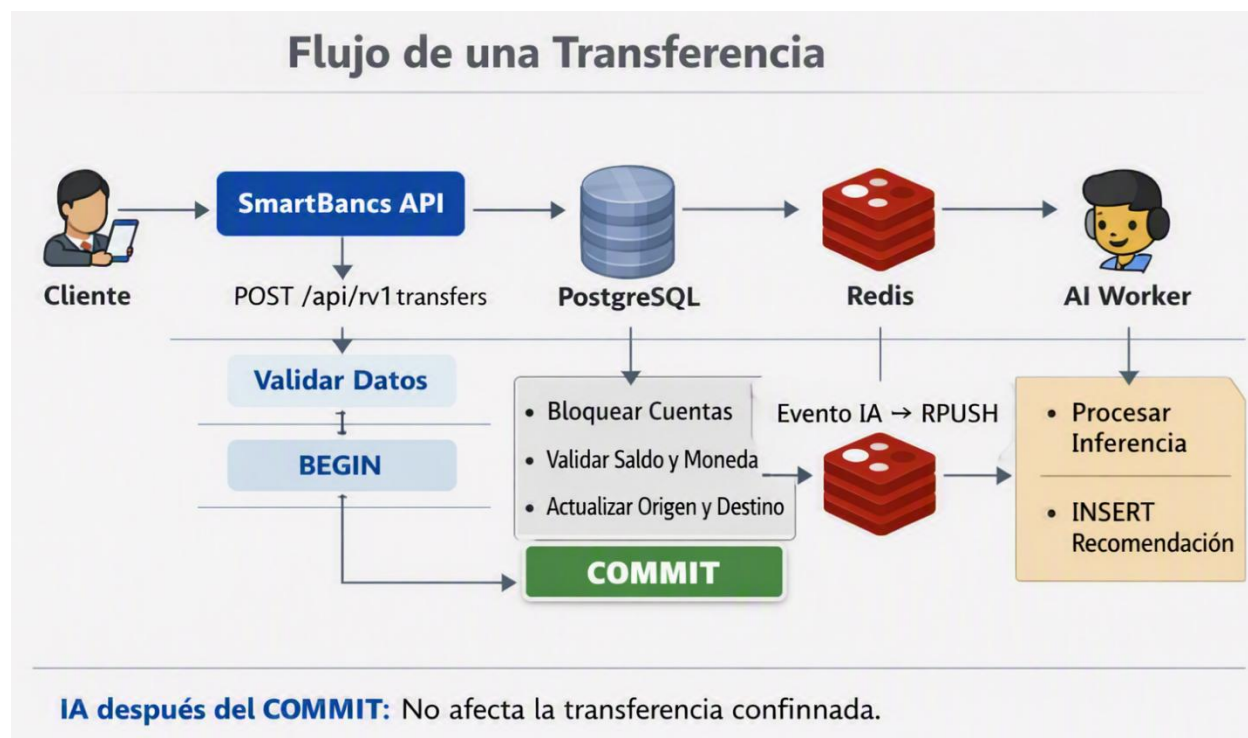
La arquitectura lógica de **SmartBancs** muestra cómo las distintas capas del sistema trabajan de forma coordinada para ofrecer servicios bancarios eficientes y seguros. El flujo comienza con el **Cliente o App**, que envía sus solicitudes al **Balanceador**, encargado de distribuirlas entre las instancias de la **SmartBancs API**. Estas APIs, desarrolladas en **FastAPI**, validan las operaciones, ejecutan las transferencias y publican eventos hacia el módulo de inteligencia artificial. La información se almacena en **PostgreSQL**, mientras que **Redis** actúa como una cola intermedia que desacopla el procesamiento transaccional del análisis de IA.

El **Servicio IA / Worker** procesa las tareas de manera independiente, aplicando reglas deterministas para generar recomendaciones o verificaciones sin depender de servicios externos. En paralelo, el **Bancs Core legado** se integra mediante un **Adaptador**, que normaliza los eventos y alimenta los módulos de **Auditoría** y **Modelo Operativo**, los cuales retroalimentan las APIs. Todo el ecosistema se monitorea mediante **Prometheus**, **OpenTelemetry** y logs estructurados en JSON, garantizando observabilidad, trazabilidad y estabilidad operativa en cada interacción del sistema.

Componentes

- **API SmartBancs:** Servicio REST stateless desarrollado con FastAPI. Valida la solicitud, ejecuta la transferencia y publica el evento de IA.
- **PostgreSQL:** Fuente transaccional del MVP. Mantiene cuentas, transferencias, recomendaciones y auditoría.
- **Redis:** Cola ligera para desacoplar el flujo transaccional del procesamiento de IA.
- **Servicio IA:** Worker independiente. En el MVP utiliza reglas deterministas para demostrar el contrato de integración sin depender de una API externa de pago.
- **Bancs:** Se considera el core legado y fuente autorizada para la reconciliación financiera. El MVP implementa la recepción y normalización de eventos.
- **Observabilidad:** Prometheus recolecta métricas; OpenTelemetry genera trazas; los servicios emiten logs estructurados en JSON.

3.2 Flujo de una transferencia



Este flujo describe cómo se procesa una transferencia desde que el **Cliente** envía la solicitud hasta que se registra en la base de datos y se genera una recomendación por parte del **AI Worker**. Primero, la **SmartBancs API** valida el payload y bloquea las

cuentas en **PostgreSQL** para asegurar consistencia. Luego verifica saldo y moneda, actualiza las cuentas de origen y destino, inserta la transacción con auditoría y confirma con un **COMMIT**.

Una vez confirmada la operación, la API envía un evento a **Redis**, que lo entrega al **AI Worker**. Este procesa la inferencia o recomendación y la guarda en la base de datos. La decisión clave es que la IA ocurre **después del COMMIT**, lo que significa que cualquier retraso o caída del servicio de IA no afecta ni revierte una transferencia ya confirmada.

4. Concurrencia y consistencia

La operación financiera utiliza una única transacción de base de datos.

El patrón utilizado es:

1. Obtener las dos cuentas.
2. Ordenar sus identificadores.
3. Bloquear ambas filas con FOR UPDATE.
4. Validar existencia, moneda y saldo.
5. Debitar la cuenta origen.
6. Acreditar la cuenta destino.
7. Registrar la transacción y el evento de auditoría.
8. Confirmar con COMMIT.

El orden determinista es importante. Si dos transferencias simultáneas involucran las mismas cuentas en sentidos opuestos, ambas intentan adquirir los locks en el mismo orden lógico, reduciendo la posibilidad de un deadlock.

Equivalente conceptual en SQL Server

En SQL Server se puede utilizar un patrón con UPDLOCK y ROWLOCK para seleccionar y bloquear las cuentas involucradas.

```
SELECT account_id, balance  
  
FROM dbo.Accounts WITH (UPDLOCK, ROWLOCK)  
  
WHERE account_id IN (@source, @destination)  
  
ORDER BY account_id;
```

El MVP ejecutable usa PostgreSQL; el repositorio incluye además sqlserver/t-sql-schema.sql como referencia para un entorno T-SQL.

5. Integración con Bancs

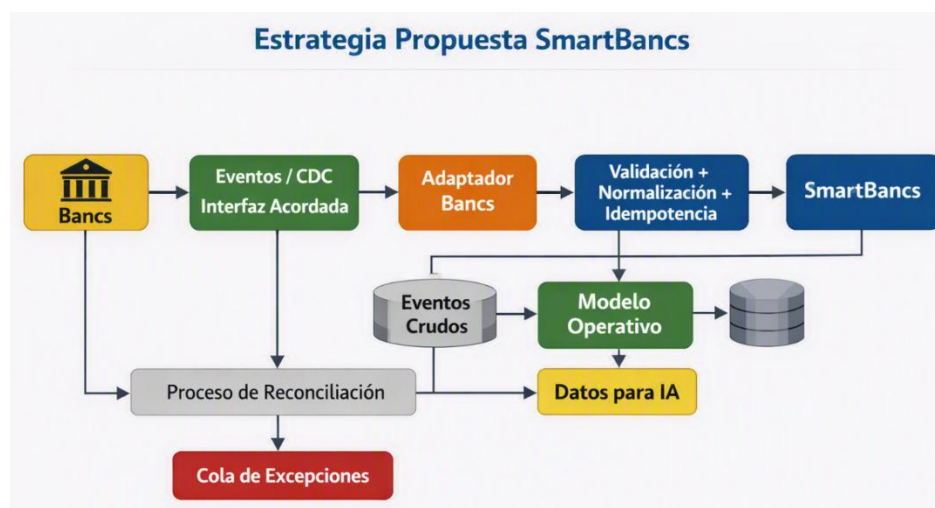
5.1 Problema

Bancs es robusto pero no está diseñado para recibir miles de consultas directas desde cada pantalla o componente de la nueva aplicación.

Consultar el core legado directamente en cada request generaría:

- mayor carga sobre el sistema crítico;
- acoplamiento entre la nueva aplicación y Bancs;
- mayor latencia;
- mayor impacto ante degradaciones del core.

5.2 Estrategia propuesta



Como se puede observar en la anterior imagen, el flujo **productivo** describe cómo el sistema **SmartBancs** transforma los eventos del core **Bancs** en información operativa y analítica. El proceso inicia cuando Bancs publica eventos mediante una interfaz acordada (CDC, cola o archivo controlado). El **Adaptador Bancs** valida el esquema, las cuentas, la moneda, la fecha y la idempotencia, preservando el evento crudo para trazabilidad y generando un evento canónico que se aplica al **modelo operativo desacoplado**. La aplicación consulta este modelo local en lugar de acceder repetidamente al core, optimizando rendimiento y disponibilidad.

En paralelo, un **proceso de reconciliación** compara saldos, totales y conteos entre el modelo operativo y Bancs, derivando cualquier diferencia a una **cola de excepciones** para revisión. El endpoint POST /api/v1/bancs/events del MVP demuestra la recepción, normalización y auditoría de los eventos sin modificar saldos, separando claramente la **recepción de datos** de la **ejecución financiera**, lo que garantiza consistencia y resiliencia en la integración.

6. Base de datos

Entidades principales

La base de datos de **SmartBancs** se estructura en torno a cuatro entidades principales que garantizan trazabilidad y consistencia en las operaciones.

ACCOUNTS

La tabla **ACCOUNTS** gestiona la información de las cuentas bancarias, incluyendo su identificador, propietario, moneda y saldo. El campo version funciona como contador de cambios de saldo, lo que permite manejar concurrencia y detectar actualizaciones simultáneas.

```
CREATE TABLE ACCOUNTS (  
    account_id VARCHAR PRIMARY KEY,  
    owner_id VARCHAR NOT NULL,  
    currency CHAR(3) NOT NULL,
```

```
balance DECIMAL NOT NULL,  
  
version BIGINT NOT NULL,  
  
created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

TRANSACTIONS

La tabla **TRANSACTIONS** registra los movimientos financieros entre cuentas. Contiene el identificador de la transacción, las cuentas origen y destino, el monto, la moneda y el estado, además de las fechas de creación y finalización para asegurar trazabilidad completa.

```
CREATE TABLE TRANSACTIONS (  
  
    transaction_id UUID PRIMARY KEY,  
  
    source_account VARCHAR REFERENCES ACCOUNTS(account_id),  
  
    destination_account VARCHAR REFERENCES ACCOUNTS(account_id),  
  
    amount DECIMAL NOT NULL,  
  
    currency CHAR(3) NOT NULL,  
  
    status VARCHAR NOT NULL,  
  
    created_at TIMESTAMPTZ DEFAULT NOW(),  
  
    completed_at TIMESTAMPTZ  
  
);
```

RECOMMENDATIONS

La tabla **RECOMMENDATIONS** almacena las sugerencias generadas por el motor de IA, vinculándolas con la transacción que las originó. La unicidad de `source_transaction_id` evita recomendaciones duplicadas para una misma operación.

```
CREATE TABLE RECOMMENDATIONS (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    account_id VARCHAR REFERENCES ACCOUNTS(account_id),  
    source_transaction_id UUID UNIQUE REFERENCES  
TRANSACTIONS(transaction_id),  
    recommendation TEXT NOT NULL,  
    model_version VARCHAR NOT NULL,  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

AUDIT_EVENTS

La tabla **AUDIT_EVENTS** conserva los eventos de auditoría relacionados con cada transacción. Permite registrar el tipo de evento, detalles en formato JSON y la fecha de creación, garantizando transparencia y trazabilidad.

```
CREATE TABLE AUDIT_EVENTS (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    transaction_id UUID REFERENCES TRANSACTIONS(transaction_id),  
    event_type VARCHAR NOT NULL,  
    details JSONB,  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

7. ETL

El script `etl/transform_transactions.py` recibe el lote crudo, realiza limpieza y genera un dataset estructurado.

Controles aplicados:

- estandarización de identificadores;
- normalización de moneda;
- conversión de montos;
- tratamiento de nulos;
- validación de fechas;
- validación de montos positivos;
- separación de registros válidos y rechazados;
- generación de un reporte de calidad.

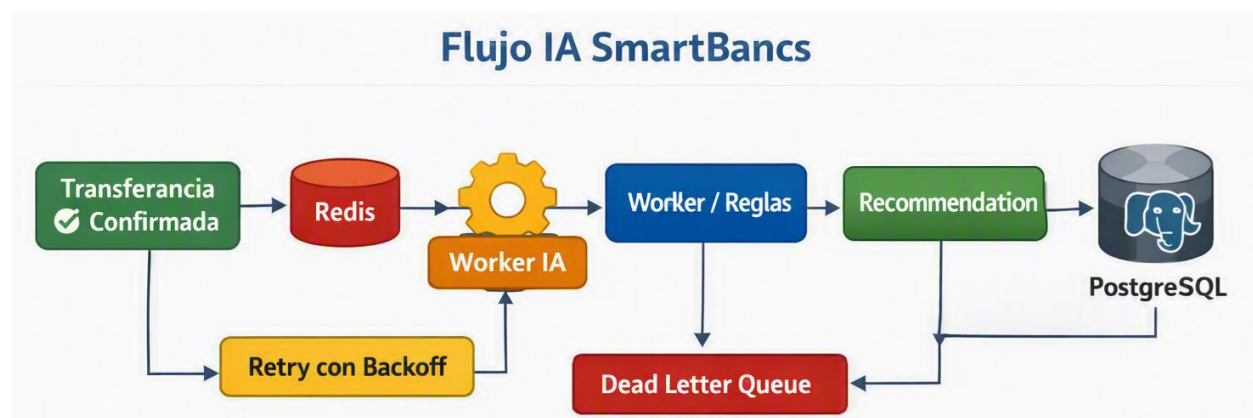
La evidencia disponible en el repositorio registra:

- 5 filas procesadas;
- 1 válida;
- 4 rechazadas.

Esto demuestra que el ETL no oculta registros inválidos: los identifica y reporta.

8. Inteligencia Artificial

8.1 Integración



El flujo comienza con una **transferencia confirmada**, que genera un evento en **Redis**. Este evento es entregado al **Worker IA**, el cual aplica el **modelo o reglas configuradas**. A partir de ello se genera una **recomendación**, que se almacena en **PostgreSQL**. El worker cuenta además con un mecanismo de **retry con backoff** que reinyecta el evento en Redis en caso de fallo, y una **Dead Letter Queue** para manejar eventos que no pudieron procesarse tras múltiples intentos.

Flujo productivo

1. Bancs publica eventos mediante una interfaz acordada, CDC, cola o archivo controlado.
2. El adaptador valida esquema, cuentas, moneda, fecha e idempotencia.
3. El evento crudo queda preservado para trazabilidad.
4. El evento canónico se aplica al modelo operativo desacoplado.
5. La aplicación consulta el modelo local/read model en lugar de consultar Bancs repetidamente.
6. Un proceso de reconciliación compara saldos, totales y conteos con Bancs.
7. Las diferencias se derivan a una cola de excepciones para revisión.

El endpoint POST /api/v1/bancs/events del MVP demuestra la recepción, normalización y auditoría del evento, pero **no modifica saldos**, separando explícitamente la recepción de datos de la ejecución financiera.

El MVP utiliza una función determinista `recommendation_for()` con reglas simples:

- Montos altos → recomendación de presupuesto o fondo de emergencia.
- Montos medios → revisión de gastos recurrentes.
- Montos bajos → seguimiento de pequeños gastos.

El objetivo es demostrar **integración, asincronía, persistencia, reintentos y trazabilidad**, no presentar estas reglas como un modelo financiero productivo.

8.2 Resiliencia

El **Worker IA** implementa mecanismos de resiliencia que incluyen:

- Reintentos automáticos.
- Backoff exponencial.
- Límite de intentos.
- Dead Letter Queue.
- Control de model_version.
- Idempotencia en la persistencia.

En caso de que la IA falle, la **transferencia financiera permanece confirmada**, garantizando que el proceso crítico de movimiento de fondos no se vea afectado por la capa de inteligencia artificial.

9. Ciclo de vida del modelo en producción



El diagrama muestra las **diez etapas** que conforman el ciclo de vida de un modelo de inteligencia artificial en producción. El proceso inicia con la **ingesta de datos autorizados**, garantizando que las fuentes sean seguras y conformes a las políticas de privacidad. Luego se realiza la **validación y versionado del dataset**, asegurando consistencia y trazabilidad. Posteriormente, los datos se dividen en **train, validation y test**, lo que permite medir el rendimiento del modelo de forma objetiva.

El siguiente paso es el **entrenamiento reproducible**, donde se aplican configuraciones controladas para garantizar que los resultados puedan replicarse. Tras el entrenamiento, se ejecuta la **evaluación del modelo** y se registra en un **repositorio de modelos** con

su versión correspondiente. El modelo se **despliega de manera controlada** en producción y se somete a **monitoreo continuo** para detectar degradación o drift. Finalmente, se realiza **reentrenamiento** cuando los criterios definidos lo indican y se aplica **rollback** si una nueva versión afecta negativamente los indicadores acordados.

Data drift

Se compararían ventanas recientes contra un conjunto de referencia mediante PSI, KS u otra técnica aprobada.

El drift debe generar una alerta y revisión; no implica automáticamente reentrenar.

Métricas del modelo

- precisión o métricas de negocio;
- cobertura;
- recomendaciones descartadas;
- latencia p95;
- errores;
- trabajos reprocesados;
- profundidad de cola.

Recursos

- CPU;
- memoria;
- concurrencia;
- tamaño de lote;
- tiempo de inferencia;
- profundidad de Redis.

10. Observabilidad

10.1 Señales

Logs

Se registran:

- solicitudes HTTP;
- transferencias completadas;
- errores de base de datos;
- deadlocks;
- publicación de trabajos IA;
- reintentos;
- eventos Bancs;
- resultados del worker.

Los logs incluyen request_id y, cuando corresponde, transaction_id.

Métricas

El MVP expone /metrics con métricas de:

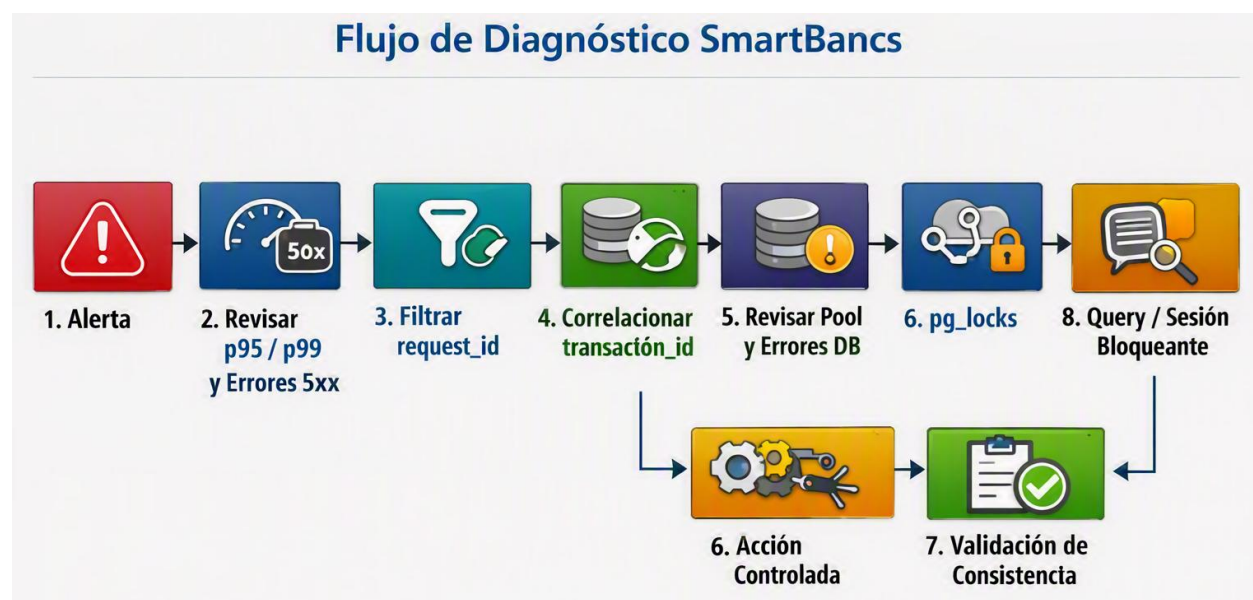
- solicitudes HTTP;
- latencia;
- transferencias;
- errores de BD;
- errores de publicación a IA;
- eventos Bancs;
- trabajos IA;
- retries;

- DLQ;
- profundidad de cola;
- journeys sintéticos.

Trazas

OpenTelemetry crea spans de la API y del procesamiento de IA, permitiendo relacionar operaciones con identificadores de solicitud y transacción.

10.2 Flujo de diagnóstico



El flujo representa el proceso sistemático para analizar y resolver alertas operativas dentro del entorno de SmartBancs. Todo comienza con una **alerta** generada por los sistemas de monitoreo. El primer paso es **revisar los percentiles p95/p99 y los errores 5xx**, que indican posibles degradaciones de rendimiento o fallos en los servicios. Luego se **filtra el request_id** para identificar la solicitud específica y se **correlaciona con el transaction_id**, permitiendo rastrear la operación afectada.

A continuación, se **revisa el pool de conexiones y los errores de base de datos**, consultando las vistas **pg_stat_activity** y **pg_locks** para detectar sesiones bloqueantes o cuellos de botella. Una vez identificada la causa, se ejecuta una **acción controlada**

para mitigar el problema y finalmente se realiza una **validación de consistencia** para asegurar que las operaciones y datos permanezcan íntegros.

Alertas configuradas

- p95 de transferencias > 2 segundos.
- Errores de base de datos.
- Backlog de IA > 100 trabajos.
- Rechazos de eventos Bancs.

11. Incidente crítico simulado

Escenario

Durante un pico de quincena aumentan la latencia, los timeouts de base de datos y los deadlocks.

Diagnóstico

Primero se determina alcance e impacto. Después se revisan p95/p99, 5xx, pool de conexiones y logs correlacionados.

Para PostgreSQL se revisaría la actividad de las sesiones mediante pg_stat_activity, incluyendo PID, usuario, estado, eventos de espera, consulta y duración.

```
SELECT pid, username, state, wait_event_type, wait_event, query,  
       now() - query_start AS duration  
FROM pg_stat_activity  
WHERE state <> 'idle'  
ORDER BY query_start;
```

También se revisarían los locks mediante pg_locks para identificar relaciones, modos de bloqueo, estado del lock y PID involucrado.

```
SELECT locktype, relation::regclass, mode, granted, pid
FROM pg_locks
ORDER BY relation, pid;
```

Acciones inmediatas

1. Declarar el incidente y establecer responsable técnico.
2. Revisar métricas y logs por request_id.
3. Identificar la consulta o sesión bloqueante.
4. Reducir tráfico no esencial.
5. Pausar temporalmente procesos no críticos que compitan por la BD.
6. Terminar una sesión bloqueante únicamente con evidencia y autorización.
7. Ajustar temporalmente pool o timeouts solo si el diagnóstico lo justifica.
8. Escalar la API si la saturación está en aplicación.
9. Validar consistencia de saldos y auditoría.
10. Mantener registro de todas las acciones.

No se recomienda matar conexiones indiscriminadamente.

12. Post-mortem

El informe post-mortem debe documentar de forma clara y verificable todo el ciclo del incidente. Inicia con un **resumen del evento**, seguido de una **línea de tiempo** que detalle los hitos desde la detección hasta la resolución. Se describe el **impacto** en los servicios y usuarios, la **forma de detección**, la **causa raíz** y los **factores contribuyentes**. También se incluyen las **consultas involucradas**, las **acciones ejecutadas**, la **evidencia recopilada** y las **acciones correctivas** propuestas. Finalmente, se asigna un

responsable, una **fecha objetivo** para la implementación de mejoras y una **prueba de no recurrencia** que valide la efectividad de las medidas.

La **prevención** se basa en prácticas de ingeniería robustas: mantener **transacciones cortas**, aplicar **orden determinista de locks**, definir **índices** adecuados y un **pool dimensionado**. Se complementa con **timeouts**, **retries con backoff** para fallos transitorios, **idempotencia**, **pruebas de concurrencia y carga**, y **alertas** sobre latencia, errores, saturación del pool y bloqueos. Este enfoque garantiza resiliencia y aprendizaje continuo ante incidentes.

13. Escalabilidad y operación

El despliegue de referencia en **Kubernetes** contempla **dos réplicas iniciales de la API**, con **readiness y liveness probes** para verificar disponibilidad. Se definen **límites y requests de CPU/memoria**, y un **Horizontal Pod Autoscaler (HPA)** que escala de **2 a 10 réplicas** según la demanda. La arquitectura incluye un **servicio independiente de IA**, con **Redis** y **PostgreSQL** como dependencias, y una **Route de OpenShift** como ejemplo de exposición externa.

La capacidad de **10 000 TPS** no se considera demostrada en entorno local; para certificarla sería necesario realizar **pruebas distribuidas** sobre infraestructura dimensionada, revisar el **límite de conexiones** de la base de datos, la **capacidad de almacenamiento**, la **red**, los **balanceadores**, los **cachés/read models** y el **comportamiento bajo fallo**. Este diseño busca garantizar escalabilidad progresiva y operación estable en entornos productivos.

14. Evidencia de rendimiento

La evidencia registrada en `evidence/benchmark-results.json` corresponde a pruebas realizadas en un entorno **Docker Desktop local**, lo que permite observar el comportamiento funcional del sistema en condiciones controladas. En la **prueba de 100 requests con concurrencia 20**, todas las solicitudes fueron exitosas, alcanzando un **p95 de 1.1713 segundos**, lo que demuestra estabilidad y cumplimiento del objetivo de rendimiento. En la **prueba de 10 000 requests con concurrencia 20**, también se

obtuvieron 10 000 respuestas exitosas, con un promedio de **96.82 RPS** y un **p95 de 0.9709 segundos**, evidenciando eficiencia en escenarios de carga moderada.

Por otro lado, la **prueba de 10 000 requests con concurrencia 1000** mostró **83 exitosas**, un **p95 de 27.2492 segundos** y **9745 errores de transporte**, lo que refleja saturación del entorno local. Esta interpretación confirma que el sistema cumple el objetivo de **2 segundos** en condiciones controladas, pero que la prueba agresiva excede la capacidad del entorno. Operacionalmente, esta evidencia es valiosa porque evita confundir el **objetivo arquitectónico de 10 000 TPS** con una **capacidad certificada**, diferenciando entre el diseño teórico y la validación práctica del rendimiento.

15. Seguridad

El MVP utiliza credenciales y datos sintéticos exclusivamente para desarrollo local.

Para producción se requerirían:

- OIDC/OAuth2;
- RBAC;
- TLS/mTLS;
- gestión centralizada de secretos;
- mínimo privilegio;
- cifrado en tránsito y reposo;
- auditoría;
- escaneo de imágenes y dependencias;
- controles de acceso a datos financieros;
- protección de información sensible;
- gestión de vulnerabilidades.